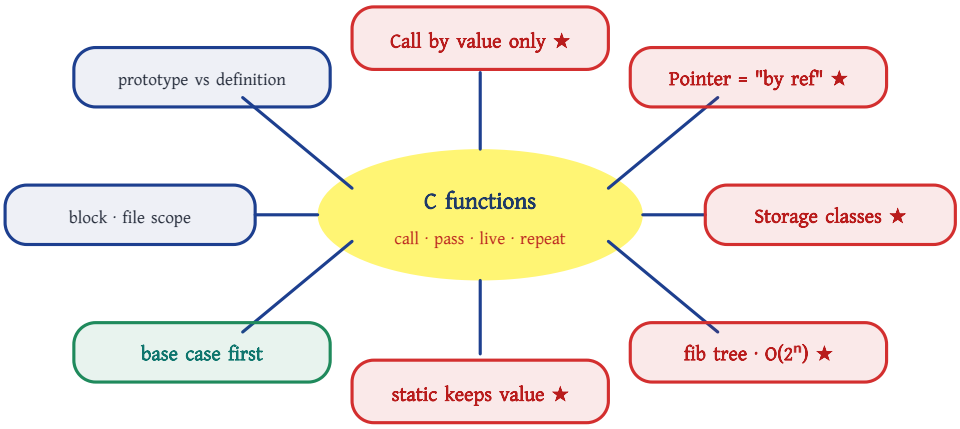
Functions, Scope & Recursion in C

Functions · Parameter passing · Scope · Storage classes · Recursion: exam-ready in 11 sheets



↑ the whole lesson on one page: red = high-yield, green = the rule every recursion hangs on

Syllabus coverage

The Structured Programming Language syllabus lists: "**Functions and program structure: parameter passing conventions, scope rules and storage classes, recursion**". Data types, operators and control structures were the previous lesson; pointers and arrays are the next, structures and unions the one after.

High-yield list

① Prototype = declaration; ends with `;`
② Actual args → copied into formal params
③ C has **only** call by value
④ "By reference" = pass a pointer
⑤ Locals default to `auto`: garbage
⑥ `static`/global default to 0
⑦ static local keeps value: 1 2 3
⑧ No `&` on a `register` variable
⑨ `extern` declares, does not define
⑩ No base case → stack overflow

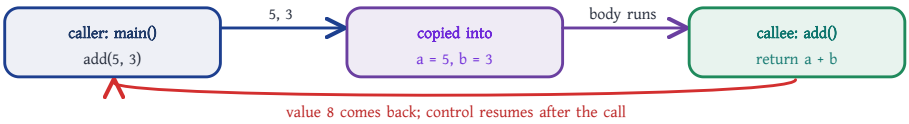
Legend: **yellow** = key term · **red circle** = must-memorise · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

Contents

✓ Functions: prototype, definition, call … p. 2
✓ Parameter passing: value vs address … p. 3
✓ Scope rules, linkage, lifetime … p. 4
✓ Storage classes: the big table … p. 5
✓ extern, static linkage, register … p. 6
✓ Recursion and the call stack … p. 7
✓ Recursion tree, complexity, iteration … p. 8
✓ Output-tracing drill … p. 9
✓ Night-before sheet … p. 10
✓ Practice MCQs + written … p. 11

1 Functions: Prototype, Definition, Call

Definition: A **function** is a named, self-contained block of code that does one job and can be called many times. A C program is a collection of functions; execution starts at `main`. General form: `return_type name(type p1, type p2) { body; return value; }`. Gives **reuse** and **modularity**. Two kinds: **library** functions (`printf`, `sqrt`) and **user-defined** functions.



one call: arguments go in, one value comes back

```c
#include <stdio.h>
int add(int a, int b);   /* prototype */

int main(void) {
    int s = add(5, 3);   /* call */
    printf("%d\n", s);   /* 8 */
    return 0;            /* success */
}

int add(int a, int b) {  /* definition */
    return a + b;
}
```

Term	Meaning
Prototype	declaration: name, return type, parameter types; ends with `;`, no body
Definition	header + body; exactly **one** per function
Call	`add(5, 3)`: jumps in, gets a value back
Actual params	arguments in the call: 5, 3
Formal params	variables in the header: *a*, *b*

Rule	What to remember
Return value	at most **one** value per call; for more, use pointer parameters or return a `struct`
`void` function	returns nothing; may use a bare `return;` to leave early
`main`	returns `int`: `0` = success; `void main()` is non-standard
Prototype	names optional (`int add(int, int);`); none at all: C89 assumed `int`, C99+ forbids it

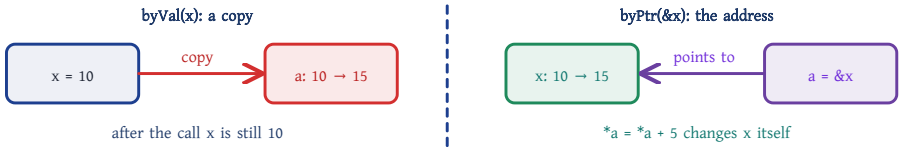
Memory trick: "**Prototype = visiting card; definition = the person.**" You may hand out many cards, but there is only one person.

Exam trap: The order in which arguments are evaluated is **unspecified**: `f(i++, i++)` is undefined behaviour. A function cannot be **defined** inside another in standard C.

Worked example ✍ output: `int sq(int x){return x*x;}` → `printf("%d", sq(3) + sq(4));` prints **25**. Actual 3, 4; formal *x*.

2 Parameter Passing: by Value and by Address

Definition: In **call by value** the callee receives a **copy** of each argument in its formal parameter, so changes inside never reach the caller. C passes **every argument by value**. To let a function change the caller's variable, pass its **address**: the formal parameter is a pointer and the function writes through `*p`. This is usually called **call by reference**, but in C it is **simulated** — the address itself is copied. True references (`int &a`) exist only in C++.



left: the function works on a photocopy · right: it holds the key to x

```
void byVal(int a)  { a = a + 5; }
void byPtr(int *a) { *a = *a + 5; }

int x = 10;
byVal(x);    /* x is still 10 */
byPtr(&x);   /* x becomes 15  */
```

```
void swap(int *p, int *q) {
    int t = *p;  *p = *q;  *q = t;
}
int a = 10, b = 20;
swap(&a, &b);  /* a = 20, b = 10 */
```

Point	Call by value	By address (pointer)	C++ reference
What is passed	copy of the value	copy of the **address**	an alias of the variable
Formal / call	`int a` / `f(x)`	`int *a` / `f(&x)`	`int &a` / `f(x)`
Caller's variable changes?	no	yes, through `*a`	yes
In C?	the **only** mechanism	yes (value of a pointer)	no — C++ only

Arrays decay to pointers: `void f(int a[])` ≡ `void f(int *a)`: only the address of element 0 is passed, the array is **not copied**, and changes inside are seen by the caller. `sizeof(a)` inside *f* = **pointer** size, so pass the length too. More: next lesson.

Structures are copied: A `struct` argument is copied whole — slow for big ones, so pass `const struct T *p`; `const` promises the data will not change. Structures in detail: lesson 4.

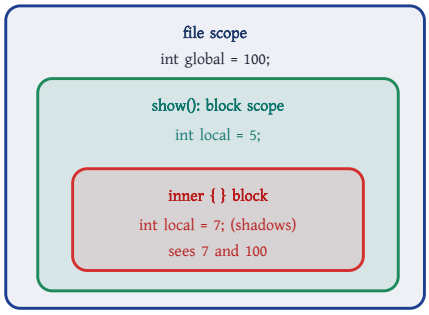
Memory trick: "**Value = photocopy; address = house key.**" Scribble on the photocopy and the original is safe; use the key and you rearrange the real house.

Common misconception: "C has call by reference" — no: the pointer is copied too. Setting `p = &y;` inside the callee does **not** move the caller's pointer.

Worked example ✍ swap by value: `void swap(int a,int b) {int t=a;a=b;b=t;}`, `x=10,y=20; swap(x,y); printf("%d %d",x,y);` → **10 20**.

3 Scope Rules, Linkage and Lifetime

Definition: The **scope** of an identifier is the part of the program text where its name can be used. A **local** (inside a block, or a parameter) has **block scope**; a **global** (outside every function) has **file scope** to the end of the file. The C standard names **four scopes**: block, file, function and function prototype.



inner boxes see outward, never inward

```
int global = 100;          /* file scope */
void show(void) {
    int local = 5;         /* block scope */
    {
        int local = 7;     /* shadows it  */
        printf("%d ", local);       /* 7 */
    }
    printf("%d %d", local, global);
}                          /* prints 7 5 100 */
/* in main(): local is unknown → error */
```

Scope (C standard)	Applies to	Visible from … to
Block	locals, function parameters	point of declaration → the closing }
File	globals, function names	point of declaration → end of the source file
Function	**labels only** (for `goto`)	the whole function, even before the label
Function prototype	parameter names in a prototype	end of that prototype

Term	Question it answers	Values
Scope	where can the name be used? (compile time)	block, file, function, prototype
Lifetime	how long does the storage exist? (run time)	automatic (block), static (whole program), allocated (`malloc…free`)
Linkage	does the same name in another file mean the same object?	**external**, **internal**, none

Memory trick: "**Scope is the room you can call the name in; lifetime is how long the person lives.**" A static local lives all program long but answers only in its own room.

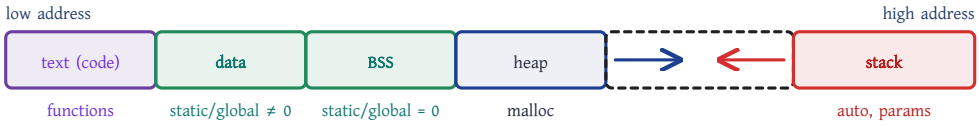
Exam trap: A shadowed global cannot be named in that block: C has no `::x` (C++ only). No use before the declaration line.

Worked example ✍ output: `int x=1; void f(){int x=2; {x=3;} printf("%d",x);}` prints **3**: the inner block assigns f's *x*; the global stays 1.

4 Storage Classes: the Big Table

Definition: A **storage class** specifier tells the compiler a variable's **storage location**, **default initial value**, **scope**, **lifetime** and **linkage**. C has four classic ones: `auto`, `register`, `static`, `extern` (C11 adds `_Thread_local`; `typedef` is one only in grammar). A local variable with no keyword is **auto**; a global with no keyword has external linkage.

Class	Stored in	Default	Scope	Lifetime	Linkage
auto	stack	garbage	block	block	none
register	CPU register (if granted)	garbage	block	block	none
static local	data / BSS	**0**	block	whole program	none
static global	data / BSS	0	file	whole program	internal
extern	wherever it is defined	0 (from the definition)	file (or block)	whole program	external
plain global	data / BSS	0	file	whole program	external



typical memory layout: where each storage class lives

```
void counter(void) {
    static int count = 0;  /* once */
    count++;
    printf("%d ", count);
}
/* main: counter(); counter(); counter();
   prints: 1 2 3                          */
/* without static:  1 1 1                 */
```

static local: three facts: ① It is created and **initialised once**, before `main` starts — the line `static int count = 0;` does **not** run again on each call. ② Its initialiser must be a **constant expression**: `static int n = f();` is a compile error in C. ③ It keeps its value between calls, yet its **scope stays the block**: `main` cannot name `count`. A function may return its address safely — it never dies.

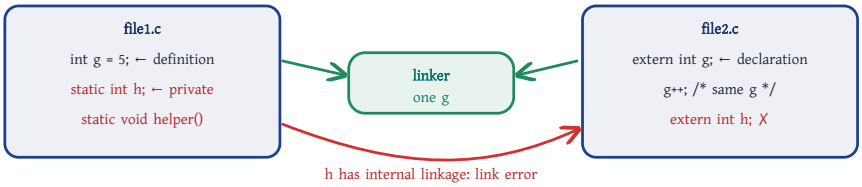
Memory trick: "**A-R die young, S-E live long.**" **A**uto and **R**egister die at the block's end with garbage; **S**tatic and **E**xtern live the whole run and start at zero.

Exam trap: Zero default = **static storage** only (static, globals). Uninitialised `auto` and `malloc` memory hold garbage (`calloc` gives 0).

Worked example ✍ output: `int f(){static int s=5; s+=2; return s;}` → `f(); f(); printf("%d", f());` prints **11** (7, 9, 11).

5 extern, static Linkage and register

Definition: `extern int g;` is a **declaration**: it says "*g* exists, defined somewhere else" and **allocates no storage**. The single **definition** `int g = 5;` outside all functions allocates the memory and carries the zero default. A program may declare *g* many times but define it **exactly once**; the **linker** joins every `extern` to that one definition.



extern reaches across files; static outside a function shuts the door

Keyword and place	Effect
`static` on a **local** variable	lifetime becomes the whole program; scope unchanged (block); no linkage
`static` on a **global** variable	**internal linkage**: visible only in its own source file
`static` on a **function**	function is private to its file (helper functions, name clashes avoided)
`extern` on a variable	declaration only; refers to an external definition
`extern int g = 5;` at file scope	an initialiser makes it a **definition** (legal; compilers warn)
functions with no keyword	`extern` by default: callable from other files through a prototype

register: A **request** to keep the variable in a CPU register for speed (loop counters). The compiler **may ignore it**, (modern optimisers choose registers themselves). Only for locals and parameters; you **cannot take &** of it — a compile error even if it ends up in memory; default value garbage.

```
void loop(void) {
    register int i;
    for (i = 0; i < 1000; i++)
        ;            /* fast counter  */
    int *p = &i;     /* ✗ error: address
                        of register   */
}
register int r;  /* ✗ not at file scope */
```

Common misconceptions — corrected: ① "`extern` creates the variable with value 0" — no: the definition elsewhere owns the storage and the zero default. ② "A `static` global can be used anywhere" — no: outside a function it means **file-private**. ③ "`register` guarantees a register" — no: it is only a hint. ④ "`&r` works on a register variable" — a compile error. ⑤ Two files both writing `int g = 1;` → **multiple definition** link error.

Memory trick: "**extern = 'lives elsewhere'; static outside = 'Keep Out' sign.**" Declaration = rumour; definition = birth.

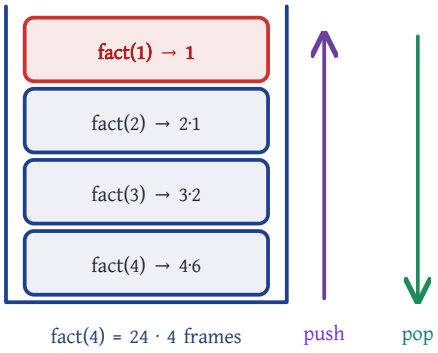
Worked example ✍ legal?: One file: `extern int n; extern int n; int n = 3;` → **legal**: two declarations, one definition.

6 Recursion and the Call Stack

Definition: **Recursion**: a function calls itself, **directly** (f → f) or **indirectly** (f → g → f). Every recursive function needs a **base case** that returns without recursing and a **recursive case** that moves towards it. Each call gets its own **stack frame** (activation record: parameters, locals, return address). With no reachable base case the frames pile up until **stack overflow** (a run-time crash).

```
long fact(int n) {
    if (n == 0 || n == 1)      /* base case */
        return 1;
    return n * fact(n - 1);    /* recursive */
}
/* fact(5) = 5*4*3*2*1 = 120 */

int fib(int n) {               /* 0 1 1 2 3 5 8 */
    if (n <= 1) return n;      /* fib(0)=0, fib(1)=1 */
    return fib(n - 1) + fib(n - 2);
}
```



winding down, unwinding up

Kind	Meaning	Example
Direct	the function calls itself	`fact`, `fib`
Indirect (mutual)	A calls B, B calls A	`isEven` ↔ `isOdd`
Tail	the recursive call is the **last** action; nothing left to do after it	`return gcd(b, a % b);`
Non-tail (head)	work remains after the call returns	`return n * fact(n - 1);`
Tree (multiple)	more than one recursive call per frame	`fib`, Towers of Hanoi

```
int isOdd(int n);          /* prototype */
int isEven(int n) {
    return n == 0 ? 1 : isOdd(n - 1);
}
int isOdd(int n) {
    return n == 0 ? 0 : isEven(n - 1);
}          /* isEven(4) → 1  (indirect) */
```

```
long factT(int n, long acc) {
    if (n <= 1) return acc;
    return factT(n - 1, n * acc); /* tail */
}
/* factT(5, 1) = 120: the frame has no
   pending work, so an optimiser can
   reuse it (a loop in disguise)      */
```

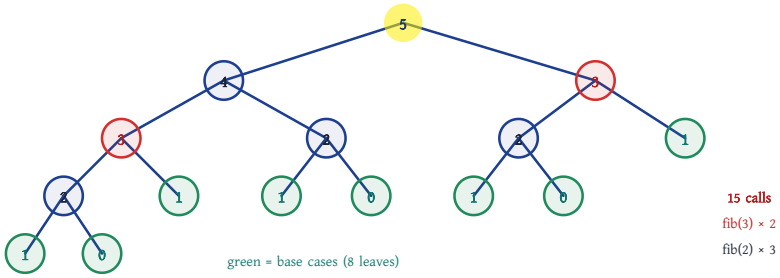
Memory trick: "**Russian dolls**": open dolls on the way down (calls), the smallest is solid (base case), close them on the way up (returns).

Exam trap: Tail-call optimisation is something compilers **may** do (e.g. gcc -O2); the C standard does not require it. `main` may call itself in C (not in C++).

Worked example ✍ depth: `fact(5)` → frames for n = 5, 4, 3, 2, 1: maximum depth **5**; result 120. `fact(-1)` never reaches the base case → overflow.

7 Recursion Tree, Complexity, Recursion vs Iteration

Definition: A **recursion tree** draws one node per call. Naive `fib(n)` makes two calls per frame and recomputes the same values again and again: calls $C(n) = C(n-1) + C(n-2) + 1$ with $C(0) = C(1) = 1$, which solves to $\mathbf{C(n) = 2 \cdot F(n+1) - 1}$. For `fib(5)`: $2 \cdot 8 - 1 = 15$ calls. Time $O(2^n)$ (tight: $\Theta(\varphi^n)$, $\varphi \approx 1.618$); stack depth only n.



node n = a call fib(n); red fib(3) is computed twice — wasted work

n	0	1	2	3	4	5	6
fib(n)	0	1	1	2	3	5	8
calls made by naive fib(n)	1	1	3	5	9	**15**	25

Algorithm	Time	Stack space
factorial, recursive	O(n)	O(n)
factorial, loop	O(n)	O(1)
fib, naive recursive	**$O(2^n)$**	O(n)
fib, memoised	O(n)	O(n)
GCD (Euclid)	O(log min(a,b))	O(log)

```
long memo[50];   /* static: all 0 */
long fibm(int n) {
    if (n <= 1) return n;
    if (memo[n]) return memo[n];
    return memo[n] =
        fibm(n - 1) + fibm(n - 2);
}         /* each n solved once */
```

Point	Recursion	Iteration (loop)
Stops at	base case	loop condition false
Memory	one stack frame per active call	constant
Speed · failure	slower (call overhead) · stack overflow	faster · infinite loop
Convertible?	yes: any recursion → loop (+ explicit stack); recursion is clearer for trees, Hanoi	

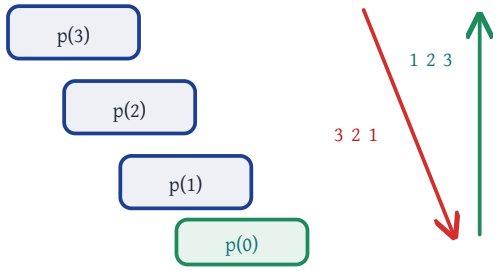
Memory trick: "**Depth is the stack, nodes are the time.**" Naive fib: n deep, about 2^n wide.

Worked example ✍ Hanoi: 3 discs need $2^3 - 1 = 7$ moves; 4 discs 15; each extra disc doubles plus one.

8 Output-Tracing Drill

Method: To trace any recursive output question: ① write the **call chain** going down with each argument; ② mark where the **base case** fires; ③ carry the **return values back up**, one frame at a time; ④ a `printf` **before** the recursive call prints on the way down, one **after** it prints on the way up (reversed order).

```
void p(int n) {
    if (n == 0) return;
    printf("%d", n);   /* before: down */
    p(n - 1);
    printf("%d", n);   /* after: up    */
}
p(3);              /* prints 321123 */
```



down prints 3 2 1, up prints 1 2 3

① Sum of digits:

```
int sd(int n) {
  return n == 0 ? 0 : n % 10 + sd(n / 10);
}
```

`sd(1234)` = 4 + 3 + 2 + 1 + 0 = **10**; 5 calls (1234, 123, 12, 1, 0).

② Power:

```
int pw(int b, int e) {
  return e == 0 ? 1 : b * pw(b, e - 1);
}
```

`pw(2, 10)` = 1024 with 11 calls. Halving *e* (`pw(b*b, e/2)`) cuts it to O(log e).

③ GCD (Euclid):

```
int gcd(int a, int b) {
  return b == 0 ? a : gcd(b, a % b);
}
```

`gcd(48, 18)`: (48,18) → (18,12) → (12,6) → (6,0) → **6**. LCM = 48·18 / 6 = 144.

④ Step of two:

```
int f(int n) {
  return n <= 1 ? 1 : n * f(n - 2);
}
```

`f(7)` = 7·5·3·1 = **105**, not 7! = 5040: the argument drops by 2.

⑤ Binary by recursion:

```
void bin(int n) {
  if (n > 1) bin(n / 2);
  printf("%d", n % 2);
}
```

`bin(10)` prints **1010**: digits come out on the way up.

⑥ static inside recursion:

```
int g(int n) {
  static int s = 0;
  s += n;  if (n > 0) g(n - 1);
  return s;
}
```

`g(3)` → one shared *s*: 3+2+1+0 = **6**; a second `g(3)` returns 12.

Memory trick: "**Print before = going down the stairs; print after = climbing back up.**" Down reads forwards, up reads backwards.

Exam trap: A `static` local is **shared by all frames** — one copy, not one per call. An `auto` local gets a fresh copy in every frame.

Worked example ✍ sum: `int s(int n){return n==0?0:n+s(n-1);}` → `s(100)` = **5050**, 101 calls deep.

Night-Before-the-Exam Sheet

Fact sheet:

Prototype: `int add(int, int);`

Definition: exactly one, with a body

Actual = in the call; formal = in the header

`main` returns `int`; `0` = success

C: call by value only

"By reference" = `f(&x)` + `int *p`

Array parameter = pointer; pass length

4 scopes: block, file, function, prototype

auto/register: garbage, block life

static/global: 0, whole-program life

static global/function → internal linkage

`extern`: declaration, no storage

No `&` on register; hint only

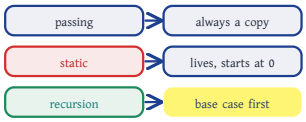
fib(5) = 5 with 15 calls; $2 \cdot F(n+1) - 1$

Top 12 one-liners for MCQs

1. Default storage class of a local: `auto`
2. static local: initialised once, keeps value
3. counter() with static ×3 → 1 2 3
4. swap by value → no change (10 20)
5. Labels have function scope
6. Shadowing: inner name hides outer
7. Recursion needs a base case
8. Each call = one stack frame
9. No base case → stack overflow
10. fact O(n); naive fib $O(2^n)$; memo O(n)
11. Tail call = recursive call is last
12. Hanoi: $2^n - 1$ moves

Top traps: C has no true call by reference · `extern` allocates nothing · static global is file-private · `register` may be ignored · `f(i++, i++)` undefined · static local shared by all frames · `sizeof` of an array parameter = pointer size.

All tricks together: visiting card vs person · photocopy vs house key · room vs lifespan · A-R die young, S-E live long · extern = lives elsewhere, static = Keep Out · Russian dolls · depth is the stack, nodes are the time · down the stairs, back up.



three ideas, three lines

Worked example ✍ one keyword changes the output: `void c(){ int n = 0; n++; printf("%d ", n); }` called three times prints "1 1 1". Write `static int n = 0;` instead and the same three calls print "**1 2 3**" — the initialiser runs once and the value survives between calls.

✓ Practice MCQs (exam pattern, 10 marks)

1. The default storage class of a local variable in C is:
(a) static (b) extern
(c) auto (d) register

2. `counter()` has `static int c = 0; c++; printf("%d ", c);`. Three calls print:
(a) 1 2 3 (b) 1 1 1
(c) 0 1 2 (d) 3 3 3

3. In C, function arguments are passed:
(a) by reference (b) by name
(c) by value and by reference (d) by value only (pointers simulate reference)

4. The default initial value of a `static int` is:
(a) garbage (b) 0
(c) 1 (d) -1

5. The address operator `&` cannot be applied to a:
(a) register variable (b) static variable
(c) extern variable (d) global variable

6. `static` on a global variable gives it:
(a) external linkage (b) block scope
(c) internal linkage (file-private) (d) automatic lifetime

7. How many calls does naive recursive `fib(5)` make, counting the first?
(a) 5 (b) 8
(c) 9 (d) 15

8. `p(n)` prints n, calls `p(n-1)`, prints n again; `p(0)` returns. `p(3)` prints:
(a) 123321 (b) 321123
(c) 321 (d) 321321

9. `extern int x;` written at file scope:
(a) declares x without allocating storage (b) defines x and sets it to 0
(c) makes x private to the file (d) allocates x on the stack

10. The time complexity of naive recursive Fibonacci is:
(a) O(n) (b) O(log n)
(c) $O(n^2)$ (d) $O(2^n)$

Answer key → 1-c · 2-a · 3-d · 4-b · 5-a · 6-c · 7-d · 8-b · 9-a · 10-d

Written question: Explain the four storage classes of C with a table (storage, default value, scope, lifetime). Write a program showing that a `static` local keeps its value, and a recursive factorial function; trace the stack for `fact(4)`. 10 marks

1. **Table (4)** — auto: stack, garbage, block, block · register: CPU register if granted, garbage, block, block, no `&` · static: data/BSS, 0, block or file, whole program · extern: declaration of a definition elsewhere, 0, file, whole program. 4
2. **static program (2)** — `counter()` with `static int count = 0; count++;` called three times prints 1 2 3; without `static`, 1 1 1. 2
3. **Factorial (2)** — base case `n == 0 || n == 1` returns 1; otherwise `n * fact(n - 1)`. 2
4. **Trace (2)** — push fact(4), fact(3), fact(2), fact(1); fact(1) = 1; pop: 2·1 = 2, 3·2 = 6, 4·6 = 24. 2

Before the exam ✓

- ✓ Can tell prototype, definition and call apart
- ✓ Can write swap by pointer
- ✓ Can name the four C scopes
- ✓ Can fill the storage-class table
- ✓ Can trace a recursive output
- ✓ Can count calls in a fib tree